

Linnæus University
Sweden

Assignment 3

Fault tolerance of Embedded System

Assignment 3



Deadline:

The deadline to submit the assignment is **19 Oct 2025**.

Rules:

1) You have to submit a report for Assignment 3, with the LNU template.
2) You have to submit a file in ***.pdf*** format.
3) You **must** use the LNU template for your report. You are allowed to use this file or a new one. In both cases, the student has to adopt the LNU template.
4) The file you submit **must be renamed** as follows <ASGM3 -surname_name.pdf>
5) Deadline is **19 Oct 2025**. Each day of delay over the submission deadline will cause a 2 points penalization.
6) The evaluation for this assignment is in points. Maximum points 50pts.

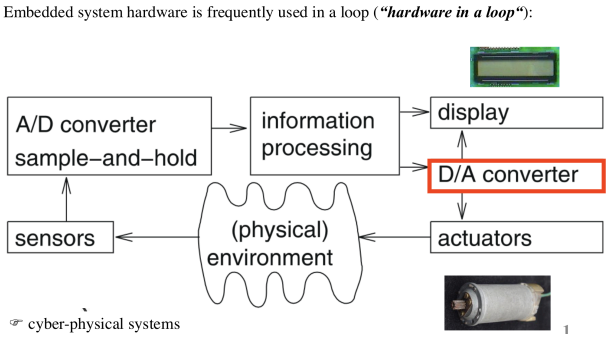
Exercise 1 (50 points)

The objective of this assignment is to investigate and understand the fault tolerance capabilities of the Raspberry Pi Pico microcontroller, identify potential failure scenarios, and develop strategies to mitigate or recover from these faults.

Initial phase:

- Conduct an initial Raspberry Pi project that includes hardware in a loop, featuring at least one sensor and one actuator. Please note that the complexity of the baseline project will influence the execution of this assignment and will also affect the final grade.

Embedded System Hardware



The designed system is primarily composed of a rain sensor, detecting droplets landing on its surface, and wiper powered by a servo wiping the droplets away upon detection. See Appendix A for instructions on how the project was set up.

- Ensure that the Raspberry Pi Pico functions correctly under normal operating conditions.

The device properly detects water upon its surface, and starts the servo window wiper. It wipes until the surface of the rain sensor has returned to not detecting rain.

Project Tasks:

Task 1: Introduction to Fault Tolerance

a) Provide a brief overview of fault tolerance, its importance in real-world applications.

The fault tolerance of a system is its ability to keep producing viable output after an error or issue has occurred, or the manner in which it stops providing that output. To have a good fault tolerance is one of the main ways allowing any system to be reliable, safe, and dependable through all the challenges that are real-world applications. [2]

b) Explain the concept of fault tolerance in embedded systems, specifically in the context of Raspberry Pi Pico.

Just like for any other system, embedded systems should have fault tolerance. Often placed out in the field, or standing as common parts of everyday life, they are very exposed to all kinds of dangers and errors. A small example of fault tolerance is having a backup interface chip for the rain sensor of the ***VindStäll*** *(Appendix A), to be employed as a safeguard if the main one gets water damage to the point of becoming unserviceable.*

Onto the fault tolerance of more specifically the Raspberry Pi Pico; The official datasheet itself states "Raspberry Pi products are not designed, manufactured or intended for use in hazardous environments requiring fail safe performance…" [3]. *The general solution for fault tolerance is to provide it with redundancy. It can be redundancy on both hardware and software level, or any mix of them both. Microcontrollers such as the Raspberry Pi Pico, and its RP2040 chip are made possible by technology shrinking as we develop new production processes, and by removing much redundancy. With that redundancy however, fault tolerance is lost. This makes fault tolerance an extra important subject when designing microcontrollers, since the fault tolerance has to be reintroduced to the system design.*

c) List the common types of faults that can occur in hardware in loop components, especially in embedded systems.

Transient Faults*; temporary malfunctions, resolved on their own. E.g. misreadings from rain sensor.*

Permanent Faults*; total component failures, remain until replaced component/other reparation. E.g. water damage to a chip, or broken/fully disconnected cables.*

Intermittent Faults*; repeated but irregular faults, remain until identified and resolved. E.g. loose connections.* [4]

Task 2: Identifying Fault Scenarios

In Task 2, you need explore potential fault scenarios that could affect your baseline project based on Raspberry Pi Pico, aiming to gain a deeper understanding of the various risks and vulnerabilities that an embedded system may encounter. This task is designed to encourage you to think critically about the potential challenges in fault tolerance and prepare you for developing relevant strategies in the subsequent tasks.

a) **Brainstorm Fault Scenarios**: you should brainstorm a list of fault scenarios. These scenarios should encompass a wide range of potential issues that could affect the operation of your baseline project. Examples include:
   - Power supply interruptions or voltage fluctuations.
   - Code errors, such as infinite loops or software crashes.
   - Hardware faults like a malfunctioning sensor or actuator or a broken connection.
   - Environmental factors like extreme temperatures or humidity.
   - Communication failures with other devices or network issues.

   *(**bold** font means the case is taken from the provided examples)*

   1. ***Power Supply loss to RPI Pico***
   2. ***Mismanaged interruptions/exceptions***
   3. ***Unexpected infinite loops in the code***
   4. *Input data out of expected range*
   5. ***Rain sensor fails to read***
   6. *Rain sensor fails to detect waterdrops*
   7. ***Servo fails to move***
   8. *Servo fails to wipe rain sensor*
   9. ***Rain sensor connection loss***
   10. ***Servo connection loss***
   11. ***Too much rainfall on sensor***
   12. *Water damage to system*
   13. *Raindrops blow away before landing on rain sensor (horizontal rainfall)*
   14. *Downfall is frozen into snow or hail*
   15. *Animals eat the wires*
   16. *Power surge to RPI Pico (too high voltage)*

b) Describe Each Scenario: For each identified fault scenario, you should provide a detailed description, explaining what the scenario entails, its potential consequences, and how it might impact the operation of the Raspberry Pi Pico.

1. ***Power Supply loss to RPI Pico:*** *The USB cable, usually providing current to the RPI Pico, suddenly stops. This could be due to a number of different factors, for example disconnecting the USB cable from the RPI Pico, or unplugging the USB cable from the wall/outlet. Doing so would fully prevent the system from continuing expected work. Normal operation should be possible to restore by simple plugging the USB cable back in, so that the power supply is restored, and the program should boot right back up.*

2. ***Mismanaged interruptions/exceptions:*** *The current code (see Appendix A.3) does not differentiate between different types of program interruptions. If program execution is interrupted by for example "ctrl+c" being run through a terminal, the program would stop after executing the* finally *block. These types of exceptions or interruptions could be solved by restarting the program.*

3. ***Unexpected infinite loops in the code:*** *The current loop logic triggering the servo if the rain sensor detects droplets, is properly set up. If the code is reconstructed improperly, there is risk of putting the* servo_wave() *call in an infinite loop, and thereby bypassing the input from the rain sensor, waving constantly instead. Do not redesign the program in such a way.*

4. *Input data out of expected range: The program currently redefines the rain sensor reading as a rain percentage, based on the highest read resistance from the sensor, updating that value during execution. If an unusually large amount of water, or some other conductor touches the rain sensor a new lowest value might be reached. This could yield unexpected and unreasonable percentage readings, possibly tricking the servo activation logic later in the program execution. Make sure the returned rain sensor values are within the, by the program defined, manageable scope.*

5. ***Rain sensor fails to read:*** *The rain sensor is currently expected to provide a reading of the format* float *or similar, to be used in calculations. If the* pin_rain.read_u16() *method reading data from the rain sensor fails, and returns an error, the program could crash during the mathematical conversion to the returned* rain percent *format (see Appendix A.3). This could be handled by controlling that the returned reading is of*

the allowed data type before making the calculations based off of it.

6. *Rain sensor fails to detect waterdrops: The rain sensor should currently detect if a droplet of water is on the surface of the sensor. If the sensor for some reason does not, e.g. because the droplet(s) does not cover a large enough portion of the sensor surface, no signal will be provided to the servo to wipe it off. This will simply make the device remain in a waiting state, until a large enough droplet is detected. This issue is difficult to manage without changing the model of the employed rainsensor to one with higher sensitivity.*

7. ***Servo fails to move:*** *Currently the servo moves a wiping arm across the surface in a controlled environment with nothing blocking the arms' movement. If something were to block the path, or hold the wiper in place, the servo would not be able to fully wipe the sensor's surface. This would mean that the system would continue to activate the servo, hoping the arm will break free enough to wipe the surface. In these attempts there's also a risk that the arm, or servo itself breaks, depending on the strength of the object blocking the wiping path. To avoid this issue, the sweeping arc could be contained a bit more, and protected from accidentally being caught on objects by a variation of guard rails.*

8. *Servo fails to wipe rain sensor: Currently wiping the wiper arm back and forth is expected to remove water droplets from its surface, returning the system to a waiting state. If the wiper fails to remove the water from the surface when the servo is activated, the system will continue to activate the servo, potentially getting stuck in a loop of continuous fruitless wiping. To avoid this you could either have a variation of a timer stopping the servo after a while upon realizing that the wiping is not fulfilling its purpose, or try to ensure that the wiper cannot fail to remove the water, e.g. by employing a backup wiper, or better surface contact by the arm. (see the video in Appendix B). This contact between the arm and rain sensor surface can be greatly assisted by creating a support structure, holding the sensor and servo at appropriate angles, and an arm prolongation for the servo to make it able to wipe a larger section of the sensor surface.*

9. ***Rain sensor connection loss:*** *The rain sensor is currently the source of triggering signals for the servo to wipe the sensor surface. If the rain sensor is disconnected, or otherwise cannot send the signals indicating droplet detection, the servo would never trigger, and the system would remain in a waiting state. This is slightly difficult to resolve without being able to regularly ping the rain sensor and update a watchdog timer for reading updates.*

10. ***Servo connection loss:*** *The servo is what moves the arm across the rain sensor surface. If the servo loses connection to the rest of the system, the wiping arm will not be moved, and therefore no water will be wiped from the sensor. This will get the system stuck in a loop of calling for the servo, and yet no wiping will be completed. This cannot really be protected against without some kind of regular ping to the servo, and watchdog timer on the RPI.*

11. ***Too much rainfall on sensor:*** *The system as designed right now is intended to wipe a few raindrops off of the sensor, not much more. If the entire sensor gets covered in droplets, or is even submerged in water, then the wiping arm will not be enough to remove them. This will get the system into a loop of constantly wiping the sensor surface fruitlessly.*

12. *Water damage to system: Currently water droplets are intended to fall upon the rain sensor surface, to then be wiped off of it. Most other parts of the system are not water tolerant, or in any other way protected against water damage. Getting water on the RPI Pico might very well short-circuit and break the entire system. To be avoided, for the preservation of the system's continued function.*

13. *Raindrops blow away before landing on rain sensor (horizontal rainfall): The rain sensor is currently set at a fix angle of around 20% incline. If the rain falls from such a direction, or blown so strongly from the side that drops do not land on the sensor surface, then they cannot be detected by the sensor. This would lead to the wiper not triggering. This is can hardly be solved with the current board.*

14. *Downfall is frozen into snow or hail: The rain sensor detects raindrops, since water leads current. If the downfall is frozen into snow or hail, the rain sensor would not detect it, and*

therefore not trigger the servo. This is not very probable for this sensor though, since it's currently a prototype designed for indoor testing.

15. *Animals eat the wires: Wires are not strong enough to withstand dental assaults from rodents or the like. Depending on which wires are damaged a number of issues could arise in the system, probably severe wherever they occur. This can be avoided by not letting animals near the system.*

16. *Power surge to RPI Pico (too high voltage): The RPI Pico supports an input voltage of up to 5V. If the current spikes, possibly from a lightning strike, then the RPI Pico might very well be damaged beyond repair. This would completely stop the system.*

c) **Prioritize Scenarios:** Once you have a list of fault scenarios, you should prioritize them. So, in addition to describing and prioritizing scenarios, create a **Failure Modes and Effects Analysis (FMEA) table** with at least **5 identified failure modes**, showing:

- Fault description
- Effect on the system
- Severity (1–10)
- Likelihood (1–10)
- Detection difficulty (1–10)
- Risk Priority Number (RPN = Severity × Likelihood × Detection)

For descriptions of each scenario, see Task 2b (above).

Below is an FMEA table listing 6 of these scenarios, with more precise prioritizations made in accordance to source[5], and task description. Generally severity is ranked on difficulty to repair, likelihood is a pure guess and determination difficulty an estimation of visibility upon testing or examination. Some cases have received an unusually low likelihood based on the fact that the device is expected to only be used for indoor demonstrations in its current form.

Descr.	Effect	Severity	Likel.	Det. diff.	RPN
1	System shutdown, no operation	8	7	2	112
2	Difficult debugging on system crash	5	7	2	70
3	System stuck	5	3	4	60
4	Might cause faulty triggers	2	4	3	24
5	System crash, no operation	7	2	7	98
6	System misses an expected trigger	2	9	3	54
7	Sensor not wiped, high physical stress to wiper	9	5	3	135
8	Wiper constantly wipes, water stays on sensor	7	8	4	224
9	Wiper doesn’t activate, water stays on sensor	6	7	6	252
10	Water stays on sensor	6	6	6	216
11	Wiper constantly wipes, risk of fault 12	7	2	1	14
12	System might break entirely	10	6	5	300
13	System misses expected triggers	6	2	1	12
14	System misses	7	1	1	7

	expected triggers				
15	System might shut down, no operation	9	1	2	18
16	System MCU might get burnt, no operation	10	2	5	100

Task 3: Developing Fault Tolerance Strategies

In Task 3, you need to focus on developing strategies to address the fault scenarios that were identified in Task 2. You need to explore various fault tolerance mechanisms that can be applied to the Raspberry Pi Pico and decide on a strategy to implement for each specific fault scenario.

a) **Review Fault Scenarios**: Begin by revisiting the list of fault scenarios identified in Task 2. **Select two fault scenarios—one hardware fault and one software fault**—that are relevant to your project/application.
Scenarios 9, 4 and 5 (due to 5 being easily solved together with the other 2)

b) **Research Fault Tolerance Mechanisms:** For **each selected scenario, develop a fault-tolerance strategy** (briefly justify why you chose the scenario and why the strategy is appropriate). Some common mechanisms include:

- Code redundancy: Implementing backup or redundant code paths.
- Error handling: Implementing error-checking and error-handling routines in the code.
- Backup power sources: Integrating backup power supplies (e.g., batteries or supercapacitors).
- Watchdog timers: Using hardware or software timers to monitor system health.
- Sensor redundancy: Employing multiple sensors for the same measurement.

Scenario 9, due to the high RPN, and the later request to showcase the functioning system in video format (Task 4 below). It is also necessary that the wiper keeps trying to wipe the sensor, since that is part of the purpose of this system. And scenario 5, due to being an important part in showcasing the project. If it doesn't read correctly, the wipers won't be started, and the system will not function. It should also be part of common coding practice to have some error detection and proofing against crashes in your code, so I felt that it was relevant to focus extra on implementing it. To also resolve a more major issue, I chose to solve scenario 4 as well, in part due to it being so similar to 5.

*For scenario 9, the easiest way would be to check if the rain sensor is returning valid input. If it isn't, as a **fail safe**, the servo falls back to a default constant wiping pace that is much slower than usual to preserve power, and sends messages in the terminal to alert the user of the issue. Using a failsafe in this way ensures that the system will keep the rain sensor dry to the best of its ability, and if nothing else at least limit water damage on the sensor surface.*

*For scenario 5, I will in part rely on the solution to scenario 9, but also add a kind of **watchdog**, keeping track of when the rain sensor last failed to read. If that interval is too short after a second failure, the system falls back to the scenario 9 fallback solution, no longer trusting the rain sensor readings as viable input since more invalid readings can be expected.*

*For scenario 4, some standard **error handling** should ensure no faulty values go into the system. By throwing manageable errors and interpreting them properly instead of leaving the code open and vulnerable to crashes when errors might occur, the risk of the whole system going down due to transient faults inducing crashes can be mitigated.*

c) **Resource Requirements:** Identify any additional components or resources required for implementing the chosen strategy (e.g., extra sensors, capacitors, hardware watchdog timer, etc.). Ensure these resources are readily available.

No additional hardware components should be required for either of these three fault solutions. It might however restructure the main.py file in a major way.

Task 4: Practical Implementation

In Task 4, you must **implement each of the two fault-tolerance strategies** you selected in Task 3—**one for the hardware fault and one for the software fault**—on the Raspberry Pi Pico. This hands-on task gives you practical experience integrating fault-tolerance mechanisms into embedded system.

a) **Code Implementation**: you need write or modify the code to integrate the selected fault tolerance strategy into their Raspberry Pi Pico project. This may involve incorporating redundancy, error handling, watchdog timers, or other mechanisms, as chosen in Task 3.

*As per the decisions made in Task 3, a **watchdog** keeping track of the number of sequential failures was implemented, and a **failsafe** fallback mode for the system, controlled by the same was constructed. Some debugging messages, and **error handling** was also implemented by encasing more of the original code in safer manners. (See Appendix C)*

b) **Hardware Integration**: If the chosen fault tolerance strategy involves additional hardware components (e.g., backup power sources, sensors), you should connect and integrate these components with the Raspberry Pi Pico.

No additional hardware was required for any of the three strategies employed.

c) **Testing**: After implementing their chosen strategies, you should carry out tests to evaluate the effectiveness of their fault tolerance mechanisms.

***Error handling** of (primarily) values out of expected range, was mostly included in the* rain_read() *section of the new code (Appendix C), where the ADC connecting to the rain sensor returns values in the range of ~2,100 for when rain is detected, and upwards of ~53,000 when none is detected on the surface. When values way below 2,100 are received, it can be assumed that the rain sensor is disconnected. Values way above 53,000 should not be obtainable unless the sensor is broken. The read value can be visually inspected by uncommenting a printout line in the* rain_read() *method.*

To test out of bounds handling;

- *Start the program while connected to a PC*

```
>>> %Run -c $EDITOR_CONTENT

                    Rain sensor raw_read:  53261
  No raindrops detected.
```

- *Unplug the rain sensor from the rest of the breadboard (a single cable should be enough, I unplugged the data cable from the sensor chip to the RPI Pico (white in Appendix A.2))*
- *The system should return an error message notifying you that the sensor has been disconnected (note the raw_read now showing way below 2,100)*

```
                    Rain sensor raw_read:  53309
 No raindrops detected.
                    Rain sensor raw_read:  1040
          - Rain sensor disconnected.
```

- *When the sensor is reconnected, the output returns to normal (upwards of 53,000)*

```
                    Rain sensor raw_read:  1024
          - Rain sensor disconnected.
 WARNING: Running in failsafe mode due to 2 failed rea
                    Rain sensor raw_read:  53228
 No raindrops detected.
                    Rain sensor raw_read:  53196
 No raindrops detected.
```

*The **watchdog** is used to keep track of the number of sequentially failed rain sensor readings that have occurred. If 3 readings in a row are failed, it enters the failsafe mode. During program execution in failsafe mode, the watchdog's failed readings count is printed to the user to help in debugging. Each time the* rain_read() *call return None, it means the reading has failed, and as such the watchdog counter is increased. When the system receives a non-faulty reading, the watchdog timer is reset to zero.*

To test the watchdog counter:

- *If the sensor surface is dry, a reading occurs around every two seconds. If the surface is wet, it reads around every nine seconds (calculated based on* sleep() *calls in the code).*
- *Unplug, and replug the same cable as in the above test, but focus on new details. (see image below)*

```
                Rain sensor raw_read:  53293
No raindrops detected.
                Rain sensor raw_read:  896
        - Rain sensor disconnected.
                Rain sensor raw_read:  896
        - Rain sensor disconnected.
                Rain sensor raw_read:  960
        - Rain sensor disconnected.
WARNING: Rain sensor reading error detected. Entering FAILSAFE mode.
                Rain sensor raw_read:  928
        - Rain sensor disconnected.
WARNING: Running in failsafe mode due to 4 failed readings from rain se
                Rain sensor raw_read:  896
        - Rain sensor disconnected.
WARNING: Running in failsafe mode due to 5 failed readings from rain se
                Rain sensor raw_read:  896
        - Rain sensor disconnected.
WARNING: Running in failsafe mode due to 6 failed readings from rain se
                Rain sensor raw_read:  53261
No raindrops detected.
                Rain sensor raw_read:  53261
```

Note;

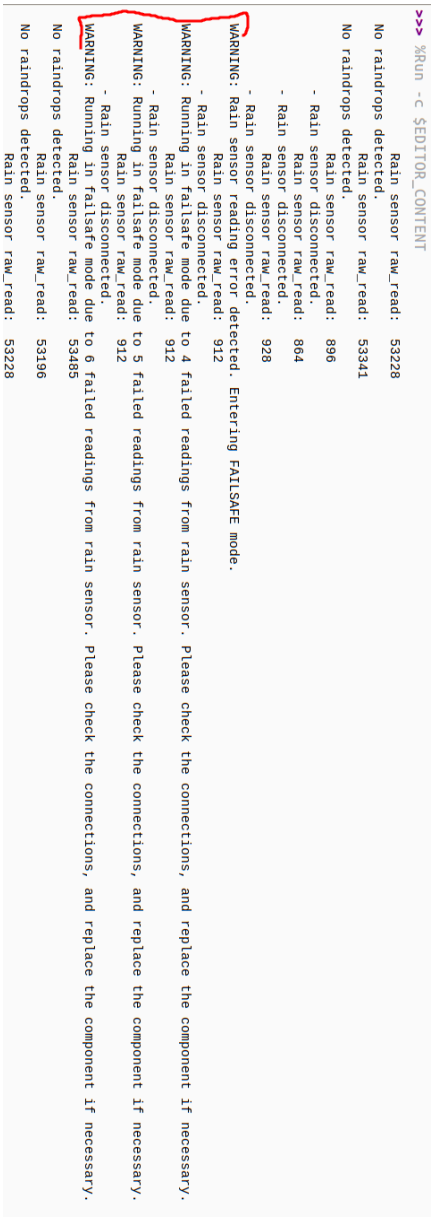
- *"Rain sensor disconnected" is returned 3 times before entering failsafe mode.*
- *In the "WARNING … due to **x** failed…" print, the number of failures is updated for each attempted reading of the sensor*

- *When the sensor is reconnected, the readings return to normal.*

*The **failsafe** functions by making the system run in a fallback mode if the rain sensor readings are deemed unusable. To run in the failsafe mode, the sensor has to continue providing unusable data (*None*) and the watchdog timer has to have reached its limit (currently it accepts only 3 invalid readings). During the failsafe mode, since the sensor can no longer control when it has to be wiped clean, the servo is running at a slower than usual, but continuous pace. This ensures that even if the rain sensor breaks, it should continue to be kept dry. While in the failsafe mode, the program repeatedly send a warning message back to the user.*

To test the failsafe mode:

- *Have the system running normally.*
- *Have an error occur with the rain sensor, and the system automatically enters the failsafe mode as soon as the watchdog counter kicks in (simulated by removing the same white cable as in the two previous tests) (see image below)*



- *As soon as the rain sensor is reconnected and provides valid readings, the system leaves the fallback mode and returns to normal functionality.*

These are all shown by essentially the same test, but different parts of the output refer to different faults having been handled (as per the above descriptions). They can all be tested at once, but to avoid the risk of missing details, I recommend testing them individually.

d) **Analysis**: you should document the results of the tests and analyse how well their strategy mitigated or recovered from the fault.

*In Appendix D I have marked the relevant sections of the feedback for each of the tests, having saved some space by running them all simultaneously (against what I just recommended in Task 4c). As can be seen by the sections marked in **red**, the program now successfully detects **out of bounds input**, and correctly interprets them as errors or faults. This successfully resolves the primary situation where fault 4 could have risen. By the sections marked in **orange**, the watchdog can be seen in action. After three **failed readings**, it triggers the failsafe mode, within it still keeping track of the number of fails for the sake of easier debugging. In **green**, the failsafe mode can be seen to have triggered upon the **rain sensor connection loss**, running until the connection to the rain sensor was restored again. When the system once again received valid input from the rain sensor, normal operation was resumed.*

*While these solutions aren't the most elegant or useful, they all manage their respective faults. They stop the system from breaking in the scenario originally described, and instead make sure that it remains operational until it can be repaired. By notifying the user via warnings in the terminal it also does its best at helping to debug, and get the attention of a user capable of performing the repairs. And while the **failsafe mode** solution doesn't resolve the connection problems, it at least mitigates the effects of rain remaining on the rain sensor surface, possibly hindering the system from being further damaged. The faulty readings received by the system cannot be fixed without managing to resolve why they were faulty in the first place, but using my implementation of the **watchdog** and **error fixing**, at least the system can remain running, and operational, if only in the failsafe mode.*

e) Prepare a short **demo video** for *Task 4: Practical Implementation.* In your **demo video**, present your work as if it were an **onsite presentation.** Clearly explain every part — the **baseline project**, the **type of fault** you are addressing, the **developed fault-tolerance strategy**, a walkthrough of your **code**, and finally, the **results.**

See Appendix B for a link to the presentation video.

Evaluation Criteria

This assignment will be assessed based on the following criteria:

- Complexity of the baseline project.
- Creativity and effectiveness of fault tolerance strategies.
- Quality of implementation.
- Comprehensive testing and evaluation.
- Clarity and organization of the report.**.** You must submit a **detailed academic-style report** that **answers each task's questions one by one in separate, clearly labeled sections** (Task 1 → Task 4). Include figures/tables where relevant, cite any sources used, and ensure the document is well organized and readable.

Notes

- This assignment should be completed individually

Sources

[1]https://micropython.org/resources/firmware/RPI_PICO_W-20250911-v1.26.1.uf2

[2]https://www.ecb.torontomu.ca/~courses/ee8205/lectures/FT-Embedded-Systems-EE8205.pdf

[3]https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

[4]https://moodle.lnu.se/pluginfile.php/10502612/mod_resource/content/1/Fault%20Tolerance-Part1.pdf (slide 10)

[5]https://quality-one.com/wp-content/uploads/2016/07/FMEA-Detections-Table.pdf

Appendices

Appendix A: Setup for the VindStäll baseline project

- ☐ *See Appendix A.1 for component list.*
- ☐ *Set up the board according to Appendix A.2*
- ☐ *Install micropython to the RPI Pico using Thonny, or via the micropython download*[1]
- ☐ *Upload the code in Appendix A.3*

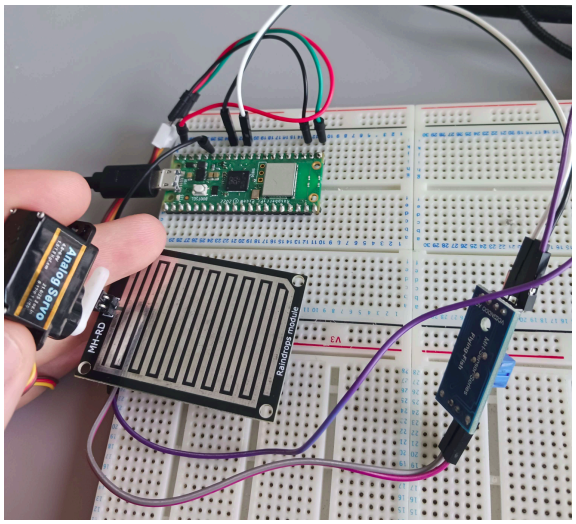
A.1

Appendix A.1: component list for the VindStäll baseline project

Component		*SEK/piece*	*Link*
Raspberry Pi Pico W	*x1*	*99 SEK*	*Electrokit*
Servo (similar)	*x1*	*~49 SEK*	*Electrokit*
Rain Sensor	*x1*	*39 SEK*	*Electrokit*
Wires			
Breadboard			
Melting glue			
USB cable			

A.2

Appendix A.2: Image of the breadboard for the VindStäll project



A.3

Appendix A.3: the main.py code for the VindStäll baseline project

```
from machine import Pin, ADC, PWM
from time import sleep

pin_led = Pin("LED", Pin.OUT)

#servo pin setup
#https://osoyoo.com/2021/07/20/raspberry-pi-pico-learnin
g-kit-lesson-5-use-potentiometer-to-control-servo/
#https://randomnerdtutorials.com/raspberry-pi-pico-servo
-motor-micropython/
pin_servo = PWM(Pin(16))
max_duty = 7864 # 0deg
min_duty = 1802 # 180deg
freq = 50
pin_servo.duty_u16(min_duty) # avoid kickbacks?
pin_servo.freq(freq)

#rain sensor setup
#https://how2electronics.com/flood-monitoring-with-raind
rop-sensor-raspberry-pi-pico/
pin_rain = ADC(0)
rain_read_max = 53200
rain_current = 0.0

def servo_wave(speed):
    pin_servo.duty_u16(min_duty)
    sleep(speed)
    pin_servo.duty_u16(max_duty)
    sleep(speed)

def rain_read():
    global rain_read_max
    amnt = pin_rain.read_u16()
    if amnt >= rain_read_max:
        rain_read_max = amnt
        #print("RECORD READING!: ", amnt)
    #else:
        #print("Read: ", amnt)
    rain_percent = (1-(amnt/rain_read_max)) * 100
    return rain_percent
```

```
pin_led.on()
sleep(1)
pin_led.off()
sleep(1)

try:
    while True:
        rain_current = rain_read()

        if rain_current > 90:
            servo_wave(0.5)
        elif rain_current > 40:
            servo_wave(1.0)
        else:
            pin_led.on()
            sleep(1.0)
            pin_led.off()
            sleep(2.0)
        print("Rain index %: ", rain_read())

finally:
        print("Program ended")
        pin_servo.deinit()
        pin_led.off()
```

Appendix B: Video of using the VindStäll v.3 (improved version)

https://drive.google.com/file/d/1MlmU0aBw5hKu4KJCmFfssN7jYJN-rDHA/view?usp=sharing

Appendix C: Updated version of main.py as per Task 4 instructions

```
from machine import Pin, ADC, PWM
from time import sleep

pin_led = Pin("LED", Pin.OUT)

# servo pin setup
#
https://osoyoo.com/2021/07/20/raspberry-pi-pico-learning
-kit-lesson-5-use-potentiometer-to-control-servo/
#
https://randomnerdtutorials.com/raspberry-pi-pico-servo-
motor-micropython/
pin_servo = PWM(Pin(16))
min_duty = 7864
max_duty = 1802
freq = 50
pin_servo.duty_u16(min_duty) # starting position
pin_servo.freq(freq)

# rain sensor setup
#
https://how2electronics.com/flood-monitoring-with-raindr
op-sensor-raspberry-pi-pico/
pin_rain = ADC(0)
is_raining = None # since errors set it to None

# Rain sensor watchdog & fail safe mode setup
sensor_watchdog = 0 # number of reading failures
allowed_fails = 3
run_failsafe = False

def servo_wave(speed: float):
    """
    Waves the wiper arm back and forth once.

    Args:
        speed (float): time time for the servo to
complete the wiping movement
    """
    pin_servo.duty_u16(min_duty)
    sleep(speed/2)
    pin_servo.duty_u16(max_duty)
    sleep(speed/2)

def rain_read() -> bool:
    """
    Reads the rain sensor value

    Returns:
```

```
        boolean: There's rain on the detector surface
(True/False)
        None: An error has occured.
    """
    try:
        raw_read = pin_rain.read_u16()
        # print("\t\tRain sensor raw_read: ", raw_read)
    except (OSError, TypeError) as e:
        print("\t- Rain sensor reading failed:", e)
        return None
    if raw_read < 1500:
        print("\t- Rain sensor disconnected.")
        return None
    elif raw_read > 60000:
        print("\t- Rain sensor most likely broken.")
        return None
    return raw_read < 4000

# Startup blinking
pin_led.on()
sleep(1)
pin_led.off()
sleep(1)

try:
    while True:
        is_raining = rain_read() # update is_raining

        # The system has just received an invalid
reading
        if is_raining is None:
            sensor_watchdog += 1

            # Rain sensor has failed, and the issue is
known by the system
            if run_failsafe is True and is_raining is
None:
                servo_wave(3.0)
                print(f"WARNING: Running in failsafe
mode due to {sensor_watchdog} failed readings from rain
sensor. Please check the connections, and replace the
component if necessary.")

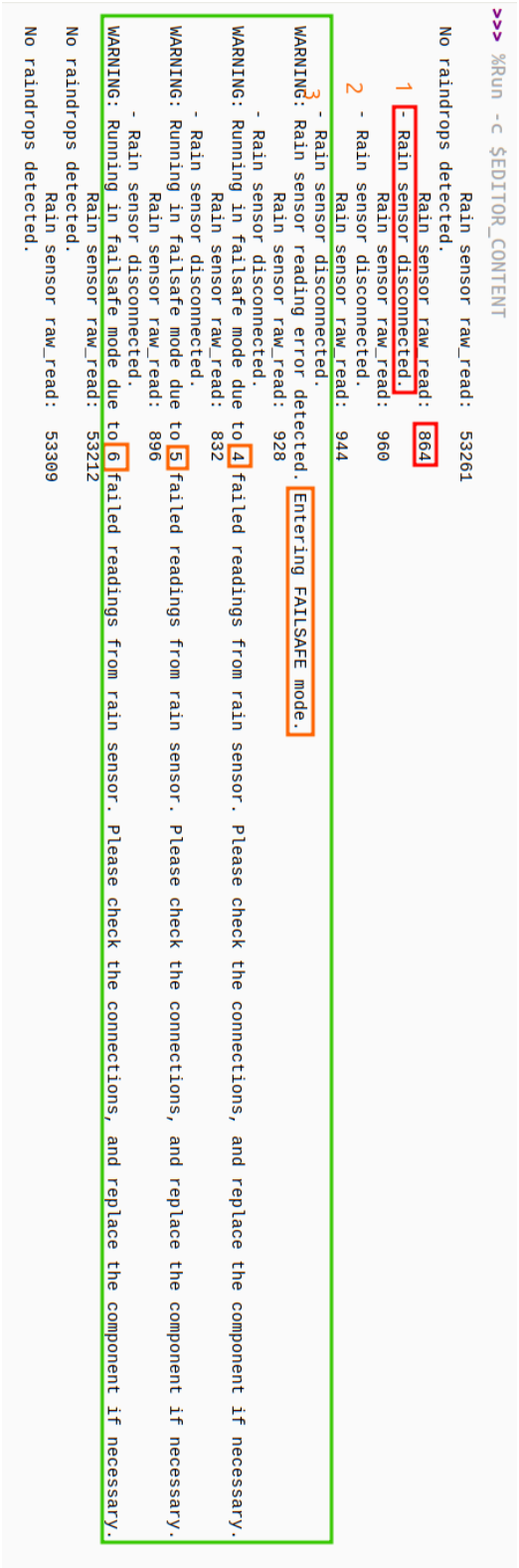
            # Error has just appeared
            elif sensor_watchdog >= allowed_fails:
                print("WARNING: Rain sensor reading
error detected. Entering FAILSAFE mode.")
                run_failsafe = True
```

```
            # The system has received is_raining
    (True/False)
            else:
                sensor_watchdog = 0

                if is_raining:
                    print("Raindrops detected.")
                    for i in range(3): servo_wave(1.5)
                else:
                    print("No raindrops detected.")
                    pin_led.on()
                    sleep(0.5)
                    pin_led.off()
                    sleep(1.0)

    finally:
            print("Program ended")
            pin_servo.deinit()
            pin_led.off()
```

Appendix D: Analysis of fault handling



Appendix E: The cad structure used for keeping VindStäll v.2 together

The stl file in question:

https://drive.google.com/file/d/1eCAg0iAPNQmH_aw5lsftp0ufIpqp-Wj4/view?usp=sharing